

The Cookie Monster in Your Browsers

The Cookie Monster in Your Browsers - @speakerdeck, filedescrptor, Speaker Deck.

- Published: 2019-08-23
- Original: <<https://speakerdeck.com/filedescriptor/the-cookie-monster-in-your-browsers?slide=26>>
- Preserved from: <https://speakerdeck.com/filedescriptor/the-cookie-monster-in-your-browsers?slide=26> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

The Cookie Monster in Your Browsers - Speaker Deck

The Cookie Monster in Your Browsers

A talk about cookies I presented in HITCON 2019

[[image: Avatar for filedescrptor](#)]

filedescriptor

August 23, 2019

More Decks by filedescrptor

[See All by filedescrptor](#)

[Exploiting the unexploitable with lesser known browser tricks](#)

[[image: Avatar for filedescrptor](#)] filedescrptor](<https://speakerdeck.com/filedescriptor>)

24

24k

[Killing ☐with ☐☐](#)

[[image: Avatar for filedescrptor](#)] filedescrptor](<https://speakerdeck.com/filedescriptor>)

7

7.4k

Other Decks in Technology

[See All in Technology](#)

[HolmesGPT](#) [SRE](#) [AI](#)

[[\[image: Avatar for Tech Leverages\]](#) [leveragestech](#)]

(<https://speakerdeck.com/leveragestech>)

PRO

0

120

[Argo CD](#) [Atlantis](#) [SRE](#)

[[\[image: Avatar for DAN\]](#) [cassius7](#)](<https://speakerdeck.com/cassius7>)

0

250

[Snowflake](#) [vol.8](#)

[[\[image: Avatar for SnowflakeLadiesGroup\]](#) [snowwmn0824](#)]

(<https://speakerdeck.com/snowwmn0824>)

0

140

[SansanTech](#)

[[\[image: Avatar for SansanTech\]](#) [sansantech](#)](<https://speakerdeck.com/sansantech>)

PRO

0

210

[AI Native Platform Engineering](#) [Platform](#) [Agile](#) [“](#) [”](#)

[[\[image: Avatar for Yuya Mizorogi\]](#) [uya116](#)](<https://speakerdeck.com/uya116>)

0

510

[Findy](#) [&](#)

[[\[image: Avatar for dip_tech\]](#) [dip_tech](#)](https://speakerdeck.com/dip_tech)

PRO

0

150

[OpenClaw](#) [Azure DevOps](#) [Wiki](#) [AI](#)

[[\[image: Avatar for yutakaosada\]](#) [yutakaosada](#)](<https://speakerdeck.com/yutakaosada>)

0

130

[Oracle Base Database Service 日本語](#)

[[\[image: Avatar for oracle4engineer\]](#) oracle4engineer]
(<https://speakerdeck.com/oracle4engineer>)

[PRO](#)

16

120k

[20260929_AmazonGuardDuty日本語AWS DevOpsAgent日本語](#)

[[\[image: Avatar for yhana\]](#) yhana](<https://speakerdeck.com/yhana>)

1

380

[20260930_Gemma4_Hands-on](#)

[[\[image: Avatar for tsho\]](#) tsho](<https://speakerdeck.com/tsho>)

0

170

[Lambda MicroVMs日本語1日本語](#)

[[\[image: Avatar for 日本語\]](#) tsukuboshi](<https://speakerdeck.com/tsukuboshi>)

1

260

[Amazon Bedrock Agents Classic日本語Amazon Bedrock AgentCore日本語2日本語](#)

[[\[image: Avatar for Kohei Matsunobu\]](#) matsunobu](<https://speakerdeck.com/matsunobu>)

0

160

Featured

[See All Featured](#)

[Design in an AI World](#)

[[\[image: Avatar for tappi\]](#) tappi](<https://speakerdeck.com/tappi>)

1

340

[How STYLIGHT went responsive](#)

[[\[image: Avatar for nonsquared\]](#) nonsquared](<https://speakerdeck.com/nonsquared>)

100

6.3k

The Curse of the Amulet

[ leimatthew05](<https://speakerdeck.com/leimatthew05>)

3

15k

Improving Core Web Vitals using Speculation Rules API

[ sergeychernyshev](<https://speakerdeck.com/sergeychernyshev>)

21

1.6k

Test your architecture with Archunit

[ thirion](<https://speakerdeck.com/thirion>)

2

2.4k

The Curious Case for Waylosing

[ cassininazir](<https://speakerdeck.com/cassininazir>)

1

550

Raft: Consensus for Rubyists

[ vanstee](<https://speakerdeck.com/vanstee>)

142

7.7k

Practical Orchestrator

[ shlominoach](<https://speakerdeck.com/shlominoach>)

192

12k

Why You Should Never Use an ORM

[ jnunemaker](<https://speakerdeck.com/jnunemaker>)

PRO

61

10k

Building Better People: How to give real-time feedback that sticks.

[ wjessup](<https://speakerdeck.com/wjessup>)

370

20k

How to build a perfect

[ jonoalderson](<https://speakerdeck.com/jonoalderson>)

1

6k

Measuring Dark Social's Impact On Conversion and Attribution

[ stephenakadiri](<https://speakerdeck.com/stephenakadiri>)

2

290

Transcript

-

The cookie monster in your browsers @filedescriptor HITCON 2019

-

@filedescriptor • From Hong Kong ! • Pentester for Cure53

• Love WebApp Sec & Browser Sec • Bug Bounty Hunter (#1 on Twitter's program)

-

Motivation

-

Motivation

-

Motivation

-

History 1966

-

The Dark Age 1994 1997 2000 Netscape's cookie_spec RFC 2109

RFC 2965 Basic Syntax Mechanism More Attributes Privacy Control Obsoletes RFC 2109
Set-Cookie2 & Cookie2 No browser followed these specs!

-

The Modern Age 2011 2015 2016 2016 RFC 6265 Cookie

Prefixes (RFC6265bis) Same-site Cookies (RFC6265bis) Strict Secure Cookies (RFC6265bis)
Obsoletes RFC 2965 Summarizes reality HttpOnly flag Improves Integrity across
subdomains over secure channel Kills CSRF & Co. Prevents secure cookies overwrite from
non-secure origin

-

None

-

None

-

**[HTTP/1.1 200 OK [...] Set-Cookie: sid=123; path=/admin
document.cookie = 'lang=en'
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f7
3171dba48e5c2/slide_10.jpg)**

HTTP Response JavaScript API (write)

-

**[HTTP/1.1 200 OK [...] Set-Cookie: sid=123; path=/admin
document.cookie = 'lang=en'
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f7
3171dba48e5c2/slide_11.jpg)**

POST /admin HTTP/1.1 [...] Cookie: sid=123; lang=en HTTP Response JavaScript API (write)
Subsequent HTTP Request document.cookie // sid=123; lang=en JavaScript API (read)
*Attributes do not appear in requests

-

Set-Cookie: sid=123; path=/admin; Secure Name Value Attribute Flag Attribute Flag

Expires Max-Age Domain Path SameSite Secure HttpOnly

-

Attribute Flag Expires Max-Age Domain Path SameSite Secure HttpOnly We

will focus on these attributes in this talk

-

Domain

-

Set-Cookie: foo=bar; domain=.example.com example.com sub.example.com sub.of.sub.example.com Domain to subdomains

-

Set-Cookie: foo=bar; domain=.example.com sub.example.com example.com sub.of.sub.example.com Subdomains to subdomains

-

Set-Cookie: foo=bar; sub.example.com example.com sub.of.sub.example.com Current domain

-

None

-

None

-

None

-

Dot or no Dot? • They have no difference (old

RFC vs new RFC style) • Both widen the scope of a cookie to all (sub)domains • The correct way to limit the scope is to not have the domain attribute • Some websites add the domain attribute for all cookies • If one of the subdomains is compromised, such cookies will be leaked to unauthorized parties

-

- RFC 6265 (4.1.2.3.) "Some existing user agents treat an

absent Domain attribute as if the Domain attribute were present and contained the current host name."

-

Still isn't fixed in IE11 on Windows 7 / 8.1!

-

None

-

Cookie Bomb • Most servers have a length limit on

request headers • When this limit is exceeded, HTTP 413 or 431 is returned • Limited cookies injection can still result in client-side DoS • Domain & Expire attributes help persist the attack across (sub)domains.

-
None

-
None

-
**[https://example.com/aaa...aaa https://twitter.com/#a
https://example.com/aaa...aaa https://twitter.com/#b
https://example.com/aaa...aaa https://twitter.com/#c GET / HTTP/1.1
[...]]
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f7
3171dba48e5c2/slide_28.jpg)**

Cookie: ev_redir_a=aaa...aaa; ev_redir_b=aaa...aaa; ev_redir_c=aaa...aaa } 8kB+

-
None

Shared domains're vulnerable by design e.g. github.io

Public Suffix List • Community curated • Some domains cannot

have cookies • The same list that restricts domain=.com.tw

-
None

-
None

XSS+OAuth • Say you have a boring XSS • And

the site is using OAuth • Sounds like you can use the XSS to takeover accounts?

-

Expectation https://google.com/oauth?client_id=example HTTP/1.1 302 Found Location: <https://example.com/oauth/callback?code=123> Set-Cookie: sid=123 HTTP/1.1

302 Found Location: <https://example.com/home> Steal

-

Reality https://google.com/oauth?client_id=example HTTP/1.1 302 Found Location: <https://example.com/oauth/callback?code=123> Set-Cookie: sid=123 HTTP/1.1

302 Found Location: <https://example.com/home> Steal 1. Authorization code is single-use 2. Intermediate HTTP Redirect is transparent

-

XSS++OAuth 1. Perform Cookie Bomb Attack via XSS 2. Embed

an iframe pointing to OAuth IdP 3. It redirects to target with the authorization code 4. Server rejects the request due to large header 5. Use XSS to get the authorization code from iframe URL

-

<https://example.com> https://google.com/oauth?client_id=example

-

<https://example.com> <https://example.com/oauth/callback?code=123> `iframe.contentWindow.location.href`

-

None

-

Path & HttpOnly

-

This is a valid request True or False? POST /admin

HTTP/1.1 [...] Cookie: csrf_token=foo; csrf_token=bar

-

None

-

Cookie Tossing • Cookie key consists of the tuple (name,

domain, path) • Each cookie-key-value has their own attribute list • (Sub)domains can force a cookie with the same name to other (sub)domains • Browser sends all cookies of the same name without attributes • Server thus has no way to tell which one is from which domain/path

-

GitHub Pages used to be on *.github.com

-

None

-

Scenario • Had an XSS on ton.twitter.com where contents are

static • twitter.com uses auth_token for session ID and _twitter_sess for storing CSRF token • Could modify _twitter_sess with an attacker-known value and have site-wide CSRF • However it's protected by HttpOnly

-

HttpOnly • Cookies with this flag cannot be read/write from

JavaScript API • Safari before version 12 has a bug that allows writing to HttpOnly cookies with JavaScript API • Cookie Tossing can also help “bypass” this flag, as you can create a cookie with the same name but different key tuple

-

Expectation Name Value Domain _twitter_sess original _twitter_sess attacker's .twitter.com POST

/i/tweet/create HTTP/1.1 [...] Cookie: _twitter_sess=attackers; _twitter_sess=original authenticity_token=attacker-known

-

Reality Name Value Domain _twitter_sess original _twitter_sess attacker's .twitter.com POST

/i/tweet/create HTTP/1.1 [...] Cookie: _twitter_sess=original; _twitter_sess=attackers; authenticity_token=attacker-known

-

-RFC 6265 (5.4) 2. The user agent SHOULD sort the

cookie-list in the following order: * Cookies with longer paths are listed before cookies with shorter paths. * Among cookies that have equal-length path fields, cookies with earlier creation-times are listed before cookies with later creation-times.

-

Precedence matters • Specs do not mention how to handle

duplicate cookies • Most servers accept the first occurrence of cookies with the same name (think of HPP) • Most browsers place cookies created earlier first

-

-RFC 6265 (5.4) 2. The user agent SHOULD sort the

cookie-list in the following order: * Cookies with longer paths are listed before cookies with shorter paths. * Among cookies that have equal-length path fields, cookies with earlier creation-times are listed before cookies with later creation-times.

-

Revised Attack Name Value Domain Path _twitter_sess original / _twitter_sess

attacker's .twitter.com /i/ POST /i/tweet/create HTTP/1.1 [...] Cookie:
_twitter_sess=attackers; _twitter_sess=original authenticity_token=attacker-known

-

-RFC 6265 (6.1) Practical user agent implementations have limits on

the number and size of cookies that they can store. General-use user agents SHOULD provide each of the following minimum capabilities: o At least 4096 bytes per cookie (as measured by the sum of the length of the cookie's name, value, and attributes). o At least 50 cookies per domain.

-

Overflowing Cookie Jar • Another way to “overwrite” a HttpOnly

cookie is to remove it • Browsers have a limitation on how many cookies a domain can have • When there is no space, older cookies will get deleted • Drawback: it's not always easy to know how many cookies a victim has (tracking cookies are unpredictable)

-

More Cookie Tossing Application

-

Self-XSS to full XSS Selectively forcing attacker's session cookie on

certain paths

-

<https://attacker.myshopify.com> https://attacker.myshopify.com/admin/oauth/authorize?client_id=editor <https://script-editor.shopifycloud.com/oauth/callback?code=attackers> document.cookie='_master_udr=attackers;path=/admin/oauth' https://victim.myshopify.com/admin/oauth/authorize?client_id=editor <https://script-editor.shopifycloud.com/oauth/callback?code=victims> Login "CSRF" Re-login victim

Self-XSS in iframe executing with victim's session

-

Session Fixation Forcing attacker's session cookie with a subdomain XSS

-

<https://script-editor.shopifycloud.com> document.cookie='_flow_session=attackers;domain=.shopifycloud.com' https://victim.myshopify.com/admin/oauth/authorize?client_id=flow GET /oauth/callback?code=victims HTTP/1.1 Host: flow.shopifycloud.com Cookie: _flow_session=attackers

Force a session cookie scoped to .shopifycloud.com using XSS OAuth redirect with authorization code

-

Implementation Discrepancy

-

Multiple Cookies at Once? • We can only set one

cookie at a time in a single Set-Cookie header • However, the older specs allow setting multiple in a single Set-Cookie header

-

Cookie based XSS Exploiting limited Cookie Injection with Safari

-

-RFC 2109 (4.2.2) "Informally, the Set-Cookie response header comprises the

token Set-Cookie:, followed by a comma-separated list of one or more cookies."

-

**[Set-Cookie: foo=123; path=/admin; HttpOnly;, bar=456; Secure
GET /admin HTTP/1.1 [...]]
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f7
3171dba48e5c2/slide_66.jpg)**

Cookie: foo=123; bar=456 Works in Safari before version 10

**[https://outlook.live.com/owa/?realm=hotmail.com;, ClientId='-
alert(2)-' HTTP/1.1 200 OK [...]] Set-Cookie: realm=hotmail.com;,
ClientId='-alert(2)-' GET]
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f7
3171dba48e5c2/slide_67.jpg)**

/ HTTP/1.1 [...] Cookie: realm=hotmail.com; ClientId='-alert(2)-' window.clientId = '-
alert(2)-'; Safari sets 2 cookies

CSRF Cookie Injection Server accepting comma separated cookies

-RFC 2965 (3.3.4) “For backward compatibility, the separator in the

Cookie header is semi-colon (;) everywhere. A server SHOULD also accept comma (,) as the separator between cookie-values for future compatibility.”

[http://blackfan.ru/r/,m5_csrf_tkn=x;,domain=.twitter.com;path=/
__utmz=123456.123456789.11.2.utmcsr=blackfan.ru|utmccn=(referral)|utmcct=/
r/,m5_csrf_tkn=x POST /messages/follow HTTP/1.1 [...]] Cookie:
__utmz=123456.123456789.11.2.utmcsr=blackfan.ru| utmccn=
(referral)|utmcct=/r/,m5_csrf_tkn=x]
(https://files.speakerdeck.com/presentations/d818b3c106a14efb9f73171dba48e5c2/slide_
70.jpg)

m5_csrf_tkn=x Cookie set by Google Analytics on translation.twitter.com scoped to
.twitter.com Twitter’s server parses it as 2 cookies

Defense

Cookie Prefixes • Cookies prefixed with __Host- cannot have Domain

attribute • This prevents (sub)domains from forcing a cookie the current domain doesn't want • Cookies intended for (sub)domains are still vulnerable to Cookie Tossing • Use a separate domain for user generated assets

-

None

-

Servers must only follow RFC 6265

-

PSA: CSRF & others will be dead in 2020

-

Q&A find me on Twitter @filedescriptor

Archived from <https://speakerdeck.com/filedescriptor/the-cookie-monster-in-your-browsers?slide=26>. Rendered offline from the local Markdown copy.